

CaptionAI: Edugenai Google Gemini powered learning assistant

Project Description:

EduGenie harnesses the power of **Google Gemini AI** to deliver an intelligent and personalized learning experience for students, self-learners, and educators. The platform provides AI-driven educational assistance through multiple learning modules, including **Question Answering, Concept Explanation, Quiz Generation, Content Summarization, and Personalized Learning Recommendations**.

Powered by the **Google Gemini API**, EduGenie processes user queries to generate accurate, context-aware, and easy-to-understand educational responses in real time. Built using **FastAPI** for the backend and a modern web-based frontend, the platform offers a fast, responsive, and user-friendly learning environment. Secure API key management through **.env** files and responsible input handling ensure reliable and secure AI interactions. EduGenie empowers learners to study more effectively by providing instant explanations, interactive quizzes, concise summaries, and tailored learning guidance across a wide range of academic subjects.

Scenarios

Scenario 1: Concept Explanation

A student preparing for an examination asks, "**Explain the concept of Machine Learning in simple terms.**" EduGenie uses the Google Gemini model to generate a clear, beginner-friendly explanation with relevant examples, helping the student quickly understand the topic.

Scenario 2: Quiz Generation

A learner studying **Data Structures** requests a quiz on **Stacks and Queues**. EduGenie instantly generates multiple-choice questions with appropriate difficulty levels, enabling the learner to assess their understanding and identify areas for improvement.

Scenario 3: Content Summarization and Learning Recommendations

A user uploads or pastes a lengthy chapter on **Database Management Systems (DBMS)** and requests a summary. EduGenie produces a concise summary highlighting the key concepts and further recommends related topics and learning resources to strengthen the user's understanding.

Technical Architecture:



Description: EduGenie follows a **modular three-tier architecture** to provide fast and intelligent AI-powered educational assistance. The **Frontend**, developed using **HTML, CSS, and JavaScript**, offers a responsive and user-friendly interface where learners can submit educational queries, request concept explanations, generate quizzes, summarize study materials, and receive personalized learning recommendations.

Pre-requisites:

- **FastAPI Documentation**
- **Google Gemini API Documentation**
- **HTML, CSS & JavaScript (W3Schools or MDN Web Docs)**
- **Python Documentation**
- **Git Documentation**
- **Google AI Studio Documentation**
- **Uvicorn Documentation** (for running the FastAPI server)

Project Workflow:

Milestone 1: Google Gemini API Integration and System Architecture

Activity 1.1:

Generate a **Google Gemini API key** and configure it securely using a **.env** file.

Activity 1.2:

Research and integrate the **Google Gemini AI model** for educational tasks such as concept explanation, question answering, quiz generation, summarization, and learning recommendations.

Activity 1.3:

Design the application architecture, defining interactions between the **Frontend**, **FastAPI Backend**, and **Google Gemini API**, including module-wise routing.

Activity 1.4:

Set up the development environment by installing the required Python libraries and dependencies for **FastAPI**, **Uvicorn**, and the **Google Gemini API**.

Milestone 2: Core Functionalities Development

Activity 2.1:

Develop the core educational modules:

- Explanation Module
- Question & Answer Module
- Quiz Generation Module
- Summarization Module
- Learning Path Recommendation Module

Activity 2.2:

Implement the **FastAPI backend** to process user requests, validate inputs, and route them to the appropriate API endpoints.

Activity 2.3:

Integrate the **Google Gemini API** to generate intelligent and context-aware educational responses for each module.

Milestone 3: Backend API Development

Activity 3.1:

Develop the main backend application using **FastAPI**, creating REST API endpoints such as:

- /explain
- /qa
- /quiz
- /summarize
- /learn

Activity 3.2:

Implement prompt engineering, request handling, response generation, and error handling for all educational services.

Milestone 4: Frontend Development

Activity 4.1:

Design and develop a responsive and user-friendly interface using **HTML**, **CSS**, and **JavaScript**.

Activity 4.2:

Connect the frontend with the FastAPI backend to send user queries and display AI-generated explanations, quizzes, summaries, answers, and learning recommendations dynamically.

Milestone 5: Testing and Deployment

Activity 5.1:

Test all API endpoints and educational modules to ensure accurate responses, proper routing, and smooth user interaction.

Activity 5.2:

Deploy the application locally using **Uvicorn** and configure the environment securely using **.env** files.

Activity 5.3:

Expose the local server publicly using **Ngrok** for demonstration and external access.

Milestone 6: Conclusion

Activity 6.1:

Evaluate the overall performance of **EduGenie**, ensuring all educational modules function correctly and deliver accurate AI-generated responses.

Activity 6.2:

Document the project architecture, API endpoints, workflow, ER Diagram, testing results, and future enhancements for deployment and maintenance.

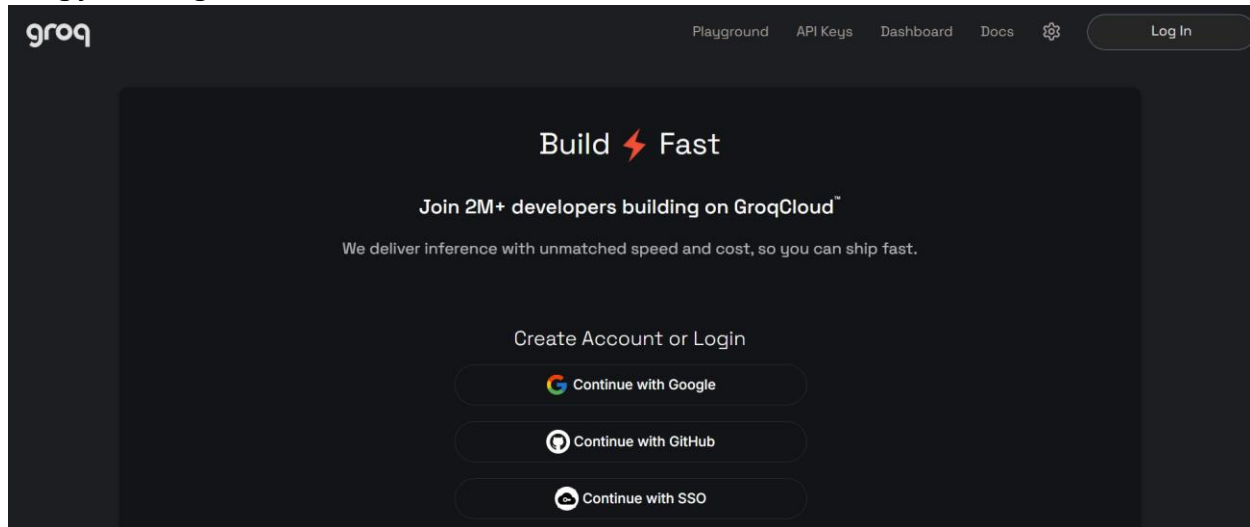
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Groq for our social media content generation needs. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating captions, hashtags, and CTAs across different platforms and tones.

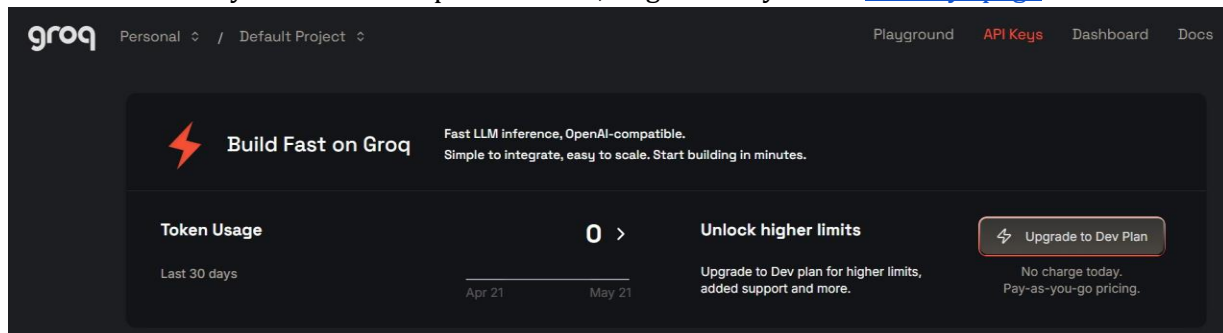
Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

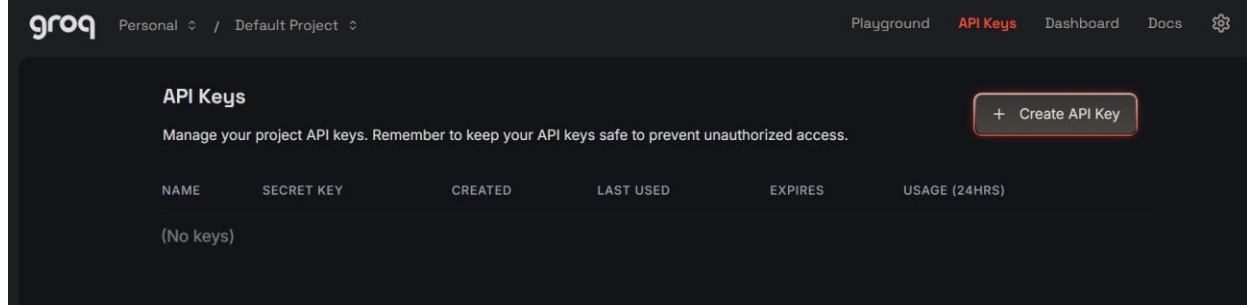
- **Step 1 — Create a Groq Account:** Visit <https://console.groq.com> and sign up for a free account using your Google account or email address.



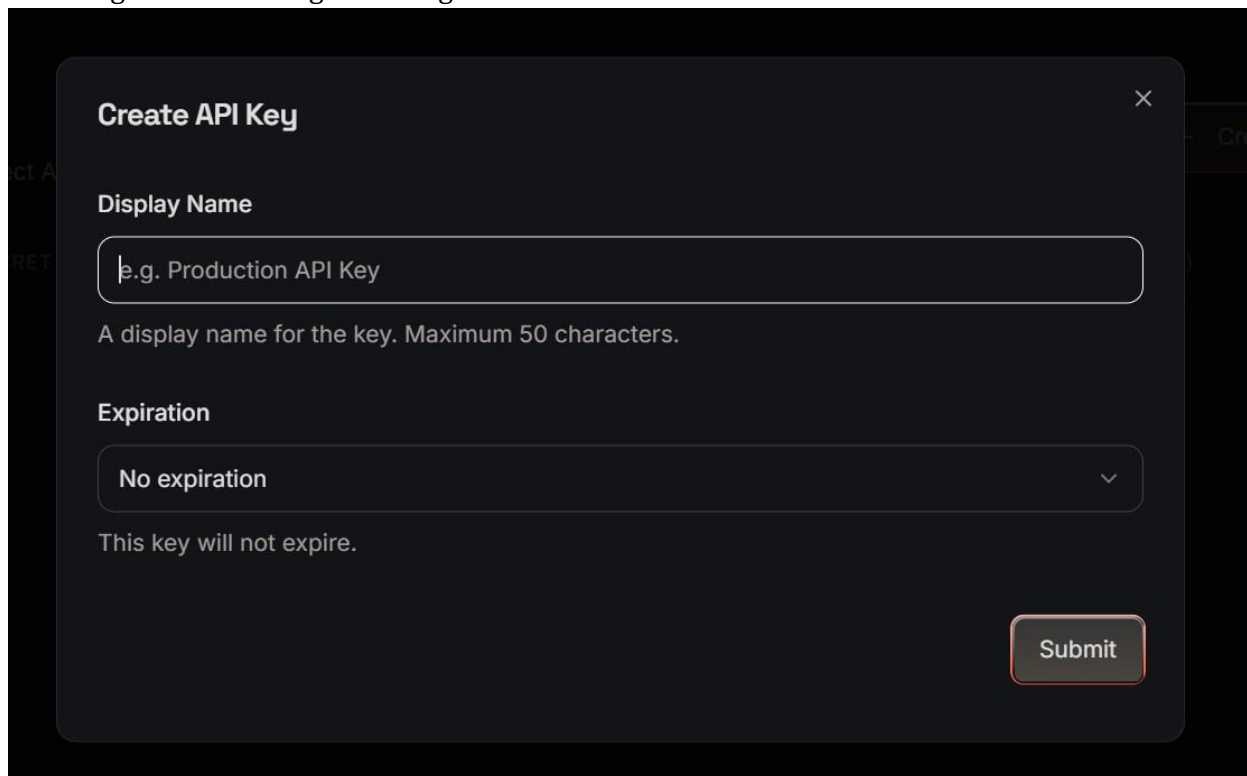
- **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select "API Keys" from the dropdown menu, or go directly to the [API Keys page](#).



- **Step 3 — Create a New API Key:** Click the “Create API Key” button. Give your key a descriptive name (e.g., captionai-key) so it is easy to identify later.



- **Step 4 — API Key:** Now give a “Display Name” and “Expiration” is optional, and Click on “Submit”. The you will get your API key, Copy it immediately and store it in a safe place you will not be able to view it again after closing the dialog.



- **Step 5 — Add the Key to Your Project:** Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below:

GROQ_API_KEY=your_copied_api_key_here

- **Step 6 — Verify the Key is Loaded:** The app.py file loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

- **Important** — Never Commit Your API Key: Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

.env

Activity 1.2: Research and Select the Appropriate Generative AI Model

The following factors were researched to select the most suitable Generative AI model for the **EduGenie AI Learning Assistant** project:

- **Understand the Project Requirements:**

The requirements of EduGenie were analyzed, focusing on educational tasks such as answering user queries, explaining complex concepts, generating quizzes, summarizing study materials, and providing personalized learning recommendations for students and educators.

- **Explore Generative AI Model Documentation:**

Different Large Language Models (LLMs) were researched, including Google Gemini models and other available open-source models. Their capabilities, context length, response accuracy, reasoning ability, speed, and limitations were evaluated for educational applications.

- **Evaluate Model Performance:**

Models were compared based on their ability to understand natural language questions, provide accurate educational explanations, generate meaningful quizzes, summarize learning content, and maintain conversational context.

- **Select the Optimal Model:**

Google Gemini 1.5 Pro was selected as the primary AI model for EduGenie because of its strong reasoning capabilities, large context window, multimodal support, and high-quality educational content generation. The model provides accurate explanations, personalized responses, and efficient handling of complex learning queries.

For lightweight tasks and efficient processing, **LaMini-Flan-T5** can also be integrated for summarization and basic text-generation tasks.

Activity 1.3: Define the Architecture of the Application

Draft an Architectural Diagram:

The architecture of EduGenie was designed to represent the interaction between different components of the application, including:

- **Frontend Layer:** HTML, CSS, and JavaScript interface for user interaction.
- **Backend Layer:** FastAPI server responsible for handling requests and application logic.
- **AI Processing Layer:** Google Gemini API integration for generating intelligent educational responses.
- **Database Layer:** Storage system for maintaining user queries, AI responses, quiz history, and learning records.

Detail Frontend Functionality:

The EduGenie frontend provides an interactive learning interface where users can:

- Enter educational questions or topics.
- Upload or provide learning materials for summarization.
- Select learning activities such as:

- Question Answering
 - Concept Explanation
 - Quiz Generation
 - Content Summarization
 - Learning Recommendations
- View AI-generated responses in a structured and user-friendly format.

The frontend communicates with the backend API to send user requests and display generated learning content.

Outline Backend Responsibilities:

The FastAPI backend manages the core functionality of EduGenie by:

- Receiving user requests through API endpoints.
- Validating and preprocessing user input.
- Creating structured prompts for the Generative AI model.
- Sending prompts to the Google Gemini API.
- Processing and formatting AI-generated responses.
- Returning structured outputs such as explanations, summaries, quizzes, and recommendations to the frontend.

Describe AI Integration Points:

The AI integration in EduGenie works through the following process:

- The backend creates customized prompts based on the user's learning requirement.
- User queries are sent to the **Google Gemini 1.5 Pro API** for intelligent response generation.
- The model analyzes the input context and generates educational content.
- The backend processes the generated response and organizes it into appropriate formats.
- The final output is displayed on the frontend for an improved learning experience.

The integration enables EduGenie to provide personalized, interactive, and AI-powered educational assistance for students and self-learners.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv  
D:\projects>"d:\projects\venv\Scripts\activate"  
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:

```
(myenv) D:\projects> pip install groq==0.11.0
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

- **Configure the API Key:** Create a .env file in the project root and add your Groq API key:
GROQ_API_KEY=your_groq_api_key_here
- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).

```
▼ static  
  > css  
  > js  
  > templates  
⚙ .env  
📄 .gitignore  
🐍 app.py  
≡ requirements.txt  
🐍 run_public.py
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the three core generation features Captions, Hashtags, and CTAs as a unified AI response driven by a single structured prompt.

- **Caption Generator:** Generates three unique, platform-appropriate captions. For Instagram/Casual tone, includes relevant emojis and conversational language. For LinkedIn/Professional tone, produces insight-led, value-driven copy.
- **Hashtag Suggester:** Produces ten hashtags per request a curated mix of high-reach popular tags and targeted niche tags relevant to the product or campaign.
- **CTA Generator:** Returns three short (under 10 words), punchy, action-oriented call-to-action phrases calibrated to the selected platform and tone.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in app.py for the home page and content generation endpoint.

Process User Input:

- Create a form in index.html for users to submit their product/campaign information, platform, and tone.
- Use Flask's `request.get_json()` to retrieve JSON data submitted from the frontend via AJAX.
- Validate that `product_info` is non-empty and within the 2000-character limit before processing.

Integrate API Calls:

- Within the `/generate` route handler, call the `build_prompt()` function to construct a structured prompt combining tone descriptions and platform-specific tips.
- Submit the prompt to the Groq API using `client.chat.completions.create()` with the `llama-3.3-70b-versatile` model.
- Parse and validate the JSON response using `extract_json()`, which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the CaptionAI application. In this milestone, you'll set up each feature's functionality through Flask routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for CaptionAI's two endpoints, each serving a distinct purpose:
 - **/** — Home page route that renders the main index.html interface

```
@app.route("/")
def index():
    return render_template("index.html")
```

- **/generate** — POST endpoint that accepts JSON input, invokes the AI, and returns structured content

```
1  import os
2  import google.generativeai as genai
3  from fastapi import HTTPException
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
9  genai.configure(api_key=GEMINI_API_KEY)
10
11  def get_gemini_response(prompt: str, temperature: float = 0.3, is_json: bool = False) -> str:
12      """Centralized function to call Gemini using the advanced 2.5-flash model."""
13      try:
14          # Upgraded to a faster, more intelligent model version
15          model = genai.GenerativeModel("gemini-2.5-flash")
16
17          config = genai.GenerationConfig(temperature=temperature)
18          if is_json:
19              config.response_mime_type = "application/json"
20
21          response = model.generate_content(prompt, generation_config=config)
22          return response.text.strip()
23
24      except Exception as e:
25          raise HTTPException(status_code=500, detail=f"Gemini API Error: {str(e)}")
```

```
1     from models.gemini_model import get_gemini_response
2     |
3     def explain_topic(topic: str) -> str:
4         prompt = f"""
5         You are an expert at breaking down complex concepts for beginners using the Feynman Technique.
6         Provide a highly engaging, beginner-friendly educational explanation for: {topic}.
7
8         CRITICAL FORMATTING RULES:
9         1. Structure your response into exactly three sections using these headers:
10            ### Simple Analogy
11            ### How it Works (The Core Mechanics)
12            ### Real World Use
13         2. Do not use any introductory text or filler. Start directly with '### Simple Analogy'.
14         3. Use formatting like bold text and bullet points to make the explanation highly readable.
15         """
16         return get_gemini_response(prompt=prompt, temperature=0.3)
```

Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads product_info, platform, and tone from the incoming JSON body using request.get_json().
- Apply input validation: return a 400 error if product_info is empty or exceeds 2000 characters.
- Route AJAX requests from the frontend to the /generate endpoint, which processes data and returns a JSON response.

Define Prompt Engineering Helpers:

- **TONE_DESCRIPTIONS** dictionary: maps professional, casual, and promotional tones to detailed copywriting instructions embedded in the prompt.

```
TONE_DESCRIPTIONS = {  
    "professional": "formal, authoritative, and business-oriented. Use clear, concise language that conveys expertise and credibility.",  
    "casual": "friendly, conversational, and relatable. Use informal language, emojis, and a warm tone that feels personal.",  
    "promotional": "exciting, persuasive, and action-driven. Use power words, urgency, and compelling offers to drive engagement."  
}
```

- **PLATFORM_TIPS** dictionary: maps instagram and linkedin to platform-specific content guidelines (character limits, style expectations).

```
PLATFORM_TIPS = {  
    "instagram": "optimized for Instagram – visually descriptive, emotionally engaging, with strong visual storytelling. Max 2200 characters.",  
    "linkedin": "optimized for LinkedIn – professional insights, value-driven, thought leadership style. Max 3000 characters."  
}
```

- **build_prompt()**: assembles a structured prompt string from the product info, selected platform tip, and tone description, instructing the model to return strictly valid JSON.

```
def build_prompt(product_info: str, platform: str, tone: str) -> str:  
    tone_desc = TONE_DESCRIPTIONS.get(tone, "engaging")  
    platform_tip = PLATFORM_TIPS.get(platform, "social media")  
  
    return f"""You are an expert social media content strategist and copywriter.
```

Integrate the Groq AI Responses in Each Function:

- Call `client.chat.completions.create()` with a system message enforcing JSON-only responses and a user message containing the assembled prompt.
- **extract_json()**: a robust parser that first attempts `json.loads()`, then strips markdown code fences (````json` blocks), and finally uses a regex to locate a raw JSON object ensuring reliable parsing across model response variations.

```
def extract_json(text: str) -> dict:
    """Extract JSON from LLM response, handling markdown code blocks."""
    # Try direct parse first
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass
    # Try extracting from markdown code block
    match = re.search(r'```(?:json)?\s*([\s\S]*)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass
    match = re.search(r'\{([\s\S]*\)}', text)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass
    raise ValueError("Could not extract valid JSON from response")
```

- Validate that the parsed result contains all three required keys — captions, hashtags, and ctas — before returning it to the frontend.
- **Home route:** `/` → renders `index.html/generate` (POST) → accepts JSON, returns `{success,captions,hashtags,ctas,platform,tone}`

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and interactive web interface for **EduGenie – Google Gemini Powered Learning Assistant**. This milestone involves creating a responsive educational dashboard with a modern glassmorphism design, allowing users to interact with the AI assistant easily. The frontend provides sections for asking questions, generating AI responses, creating quizzes, summarizing learning materials, and displaying personalized learning recommendations through dynamic content rendering using HTML, CSS, and JavaScript.

Activity 4.1: Designing and Developing the User Interface Set Up the Base HTML Structure:

- Develop the main HTML file (**index.html**) as a single-page interface for EduGenie, including:
 - Header section with EduGenie logo, title, and educational tagline.
 - User input section for entering questions, topics, or learning materials.
 - AI response section to display generated answers and explanations.
 - Additional sections for quiz generation, summaries, and recommendations.
 - Use semantic HTML5 elements such as `<header>`, `<main>`, `<section>`, and `<footer>` to maintain a structured and accessible layout.
 - Integrate:
 - **Google Fonts (Inter/Poppins)** for clean and readable typography.
 - **Font Awesome icons** for educational indicators such as book, brain, quiz, summary, and learning icons.
-

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (**static/css/style.css**) implementing a modern dark glassmorphism theme for EduGenie.
- Add animated background elements (gradient blobs and floating effects) to create an engaging AI-powered learning environment.
- Design responsive layouts using:
 - **Flexbox** for arranging input and response components.
 - **CSS Grid** for organizing AI outputs such as answers, quizzes, summaries, and recommendations.
- Implement media queries to ensure compatibility across:
 - Desktop screens
 - Tablets
 - Mobile devices

Create Separate UI Components for Each Output Type:

AI Answer Card:

- Display Gemini-generated explanations inside individual glass cards.
- Include:
 - Topic heading
 - AI-generated explanation
 - Copy response button
 - Feedback animation after copying

Quiz Section:

- Create interactive quiz cards containing:
 - Question text
 - Multiple-choice options
 - Answer reveal functionality
 - Score display

Summary Card:

- Display AI-generated summaries in a separate glass component.
- Provide:
 - Key points
 - Important concepts
 - Copy/download option

Learning Recommendation Section:

- Display personalized learning suggestions using card-based layouts.
- Include:
 - Recommended topics
 - Learning resources
 - Next learning steps

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener to the EduGenie input form.
- Prevent default page refresh behavior and send user input data as a JSON POST request to the backend API endpoint (**/generate**) using the Fetch API.
- The request includes:

- User query/topic
 - Selected learning mode (Question Answering, Quiz Generation, Summary, Recommendation)
-

Implement Loading and Response Handling:

- During AI processing:
 - Disable the submit button.
 - Display an animated loading spinner.
 - Show "EduGenie is thinking..." status message.
 - After receiving the API response:
 - Dynamically update the UI without refreshing the page.
 - Render Gemini-generated content into appropriate sections:
 - AI explanations
 - Quiz questions
 - Summaries
 - Recommendations
-

Add Interactive Frontend Features:

- Implement copy-to-clipboard functionality for AI responses.
- Add smooth scrolling to generated results.
- Provide visual feedback for successful actions.
- Handle API errors gracefully by displaying user-friendly error messages.
- Ensure the interface provides a smooth and engaging AI-powered learning experience.

Render Results Dynamically:

- Implement the `renderResults(data)` function in `main.js`, which dynamically creates and injects DOM elements for captions, hashtags, and CTAs into their respective containers.
- After rendering, automatically smooth-scroll the viewport to the results section using `scrollIntoView()`.

Restore UI State After Generation:

- In the finally block of the async handler, re-enable the submit button and restore the original button text and icon regardless of success or failure.

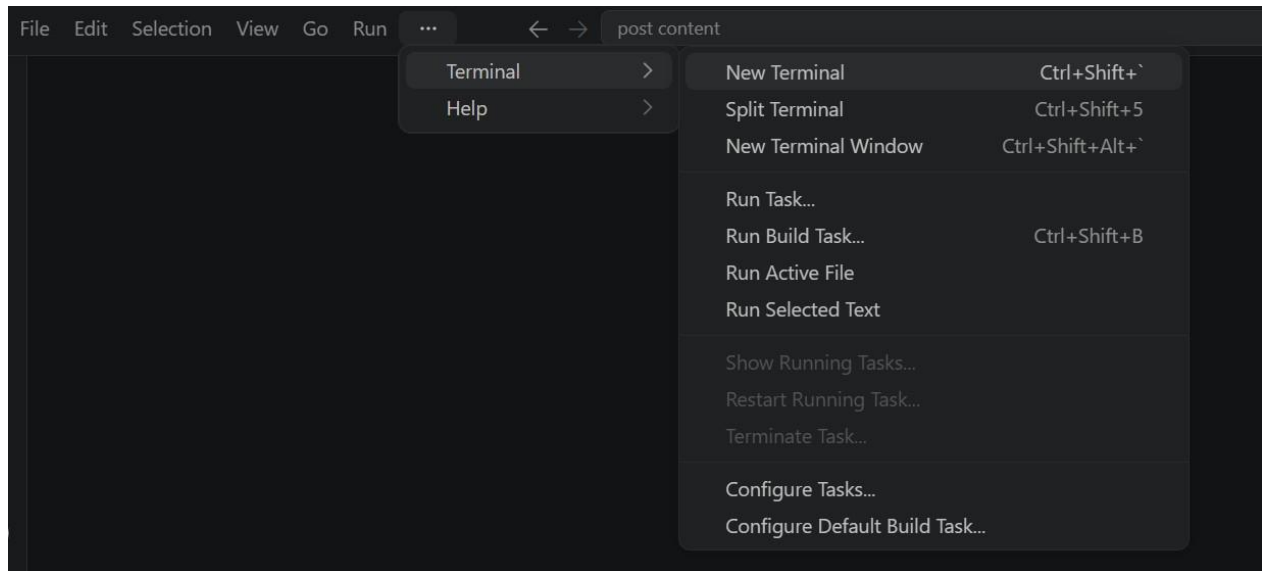
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the CaptionAI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

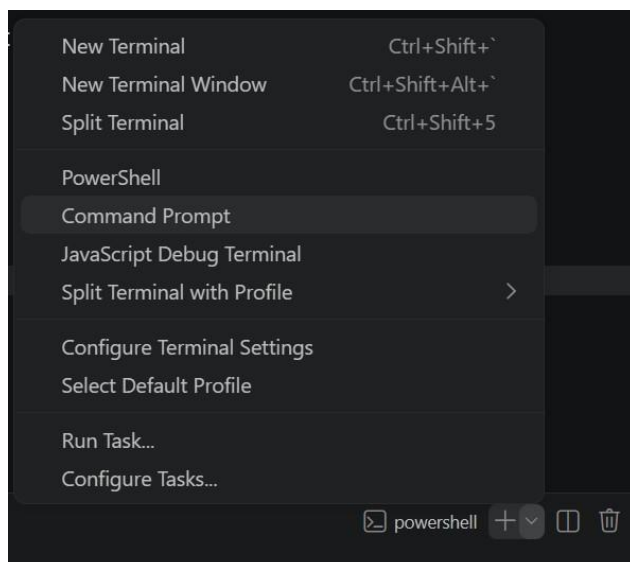
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open "Command Prompt"



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:
GROQ_API_KEY=your_groq_api_key_here

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5050" to access the app.

```
(venv) D:\projectsSB\AI in healthcare\startup-validator>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
d. Follow link (ctrl + click)
* Running on http://127.0.0.1:5050
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 442-110-195
```

- Interact with the caption generator test all three tones (Professional, Casual, Promotional) and both platforms (Instagram, LinkedIn) to verify the AI responses are correctly generated, parsed, and rendered.

Activity 5.3: Public Deployment via Ngrok

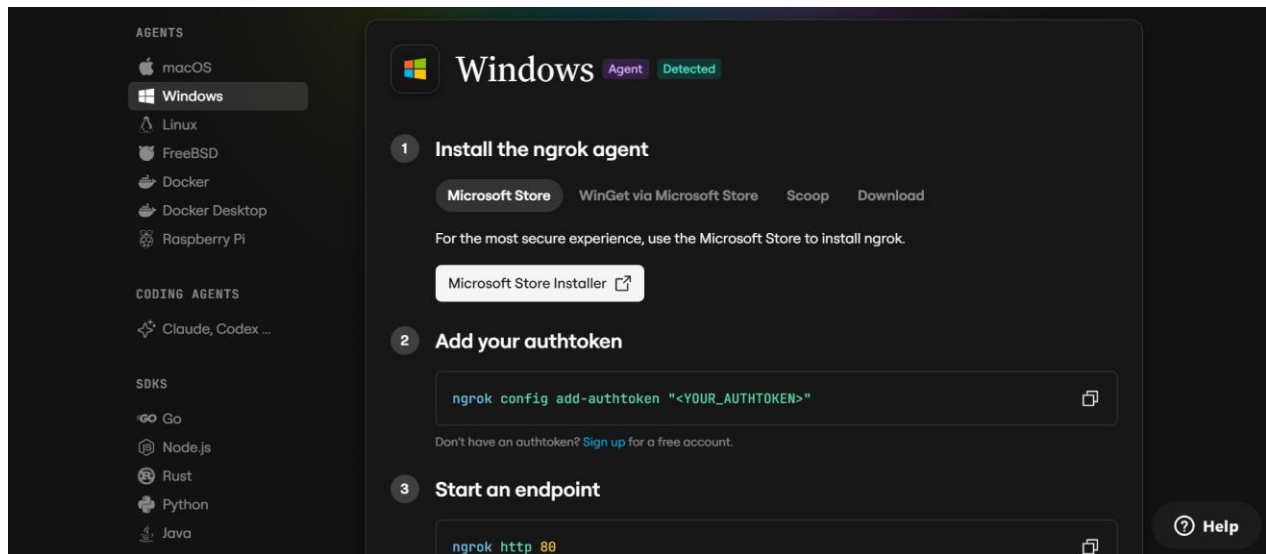
Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from <https://ngrok.com/download>.
- Click on “Microsoft Store Installer”.



Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The run_public.py script sets the authtoken automatically using:

```
D:\projects>ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Replace the TOKEN value in run_public.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running app.py directly, launch run_public.py to start both the Ngrok tunnel and the Flask server:

```
D:\projects>python run_public.py
```

- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:

```
=====
YOUR PUBLIC WEBSITE URL IS LIVE!
URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
=====
```

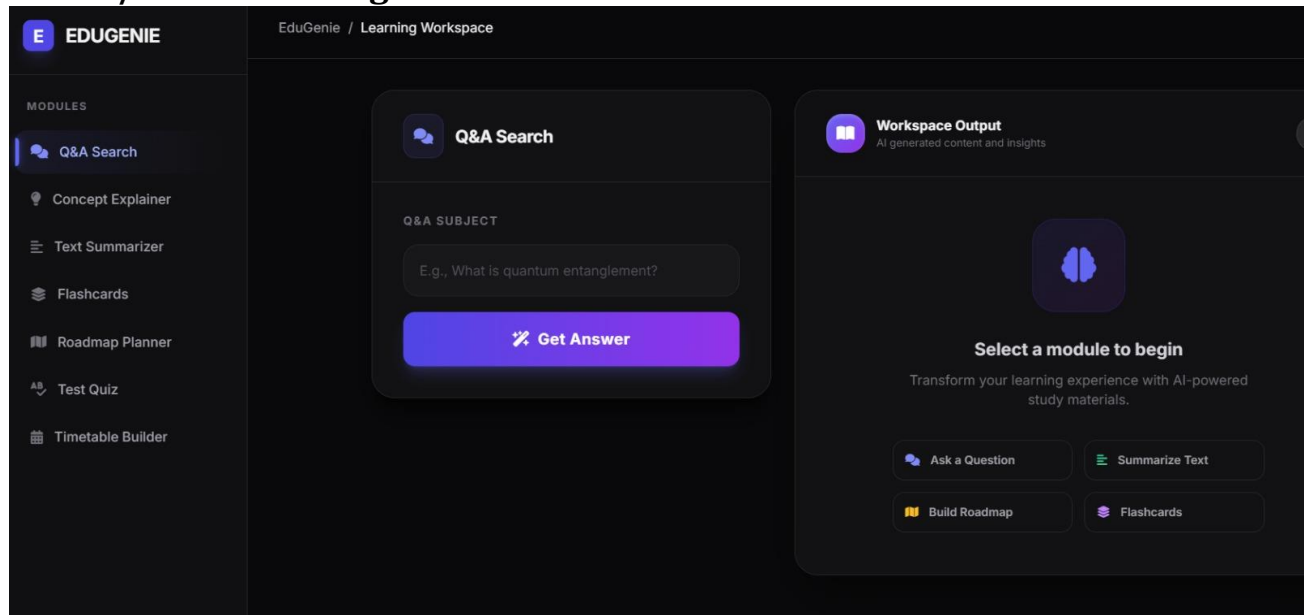
- Share this URL with anyone who can access your CaptionAI app directly from their browser without any installation.

Important Notes:

- **debug=False** is required in run_public.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart run_public.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart run_public.py to get a new URL when this happens.

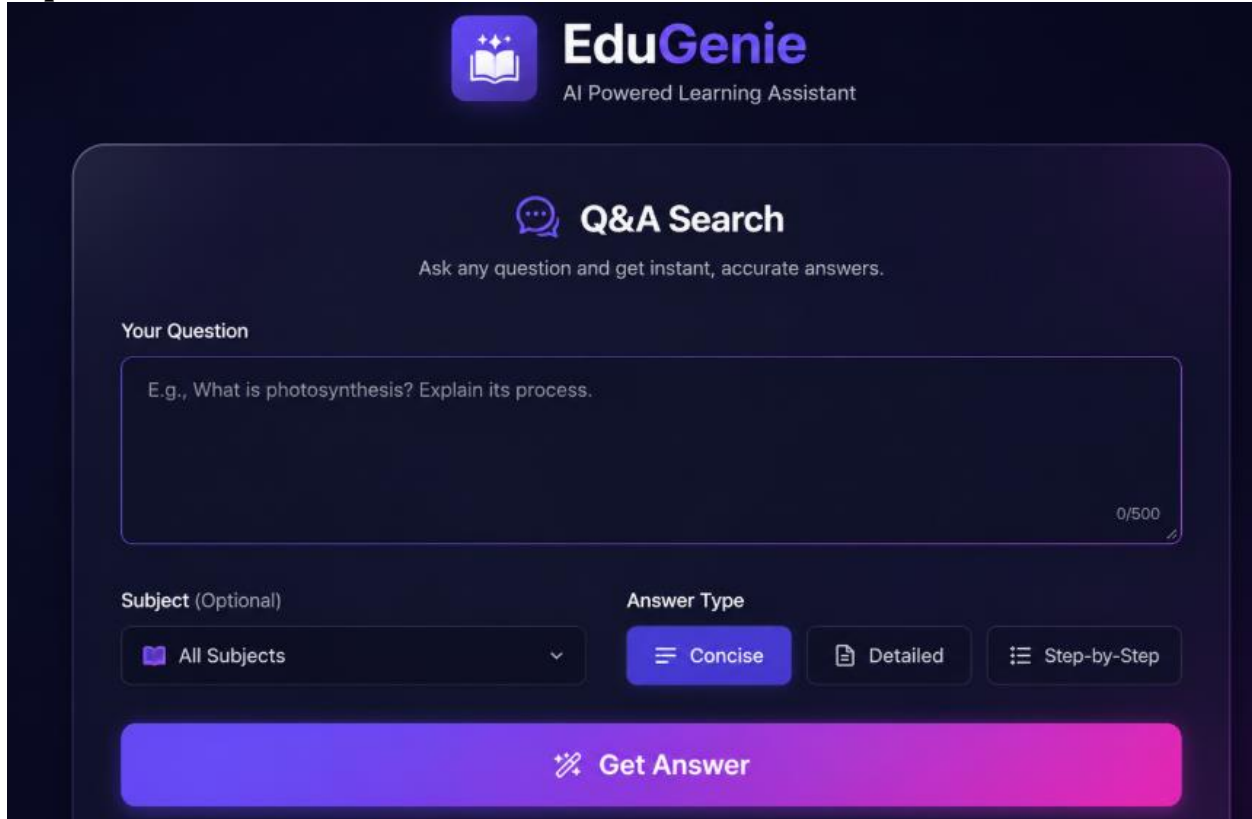
Exploring the Website's Web Pages:

Home / Generator Page:



Description: The single-page interface of **EduGenie** serves as both the landing page and the AI-powered learning assistant. It features a modern dark glassmorphism design with animated background elements, a header displaying the **EduGenie** brand (graduation cap/book icon + logo), and the tagline "**Learn Smarter with AI.**" The main section provides an intuitive educational workspace where users can enter learning queries, upload study material (optional), select an AI learning feature (such as Question Answering, Concept Explanation, Quiz Generation, Content Summarization, or Learning Recommendations), and receive intelligent AI-generated responses in a dedicated results panel. All learning interactions take place seamlessly on a single page, providing a fast, interactive, and user-friendly educational experience.

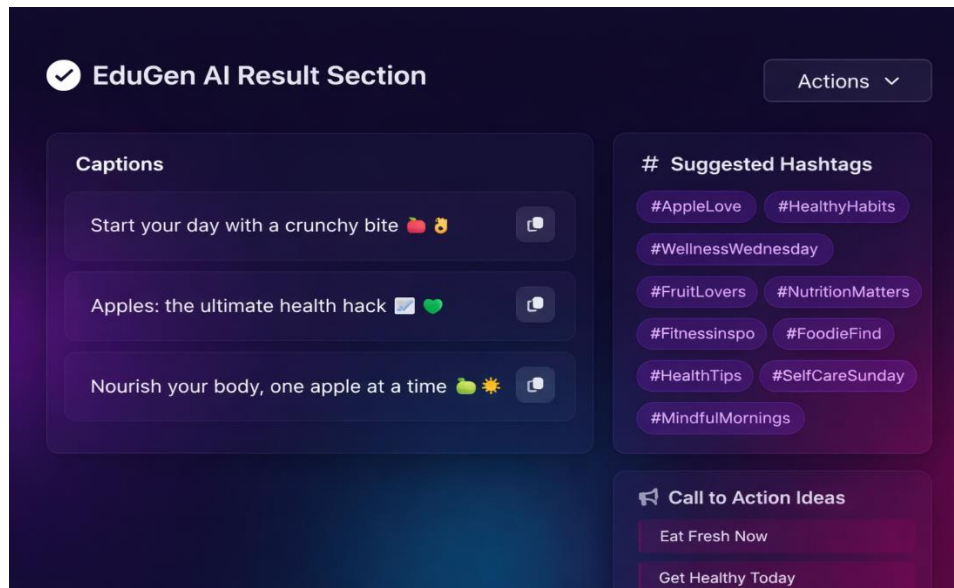
Input Section:



The interface is a dark-themed web application. At the top, it features the 'EduGenie' logo with a book icon and the text 'AI Powered Learning Assistant'. Below this is a 'Q&A Search' section with a speech bubble icon and the instruction 'Ask any question and get instant, accurate answers.' A large text input field is labeled 'Your Question' and contains the placeholder text 'E.g., What is photosynthesis? Explain its process.' with a character count '0/500'. Below the input field are two rows of controls: 'Subject (Optional)' with a dropdown menu currently showing 'All Subjects', and 'Answer Type' with three buttons: 'Concise' (selected), 'Detailed', and 'Step-by-Step'. At the bottom is a large, wide button labeled 'Get Answer' with a magic wand icon.

Description A glass-styled card contains the main AI learning interface. Users can enter their educational question, topic, or learning request in a textarea (up to 2000 characters) and optionally upload study material such as PDF, DOCX, or TXT files for context. They can then select the desired AI learning feature from options such as **Question Answering**, **Concept Explanation**, **Quiz Generation**, **Content Summarization**, or **Learning Recommendations**. After clicking the **Generate Response** button, the system processes the request using the AI model, and the button transitions into an animated loading spinner until the response is generated. The AI-generated educational content is then displayed in a dedicated results panel for an interactive and seamless learning experience.

Results Section:



Description: Revealed dynamically after generation, this section displays the **AI-generated learning response** in a structured two-column layout. The left column presents the main educational output through interactive response cards containing the generated explanation, solution, or summarized content along with a copy-to-clipboard button for easy access. The right column contains additional glass-styled cards featuring supporting learning resources such as generated quizzes, key points, important concepts, or personalized recommendations in an organized format. The viewport automatically smooth-scrolls to this section after the AI response is generated, providing a seamless and engaging learning experience.

Conclusion: EduGenie is an AI-powered learning platform built using **FastAPI** and integrated with the **Google Gemini API** to provide intelligent educational assistance. The platform streamlines the learning process by offering features such as **Concept Explanation, Question & Answering, Quiz Generation, Content Summarization, and Personalized Learning Recommendations** through a simple and user-friendly interface. By leveraging the capabilities of Google Gemini, EduGenie delivers accurate, context-aware, and real-time educational responses, helping students and self-learners understand complex topics more effectively.

The project involved AI model integration, backend API development, frontend implementation, testing, and deployment using **Ngrok**. It demonstrates how Generative AI can enhance digital education by making learning more interactive, accessible, and personalized. With its modular architecture and scalable design, EduGenie serves as a strong foundation for modern AI-assisted learning systems.